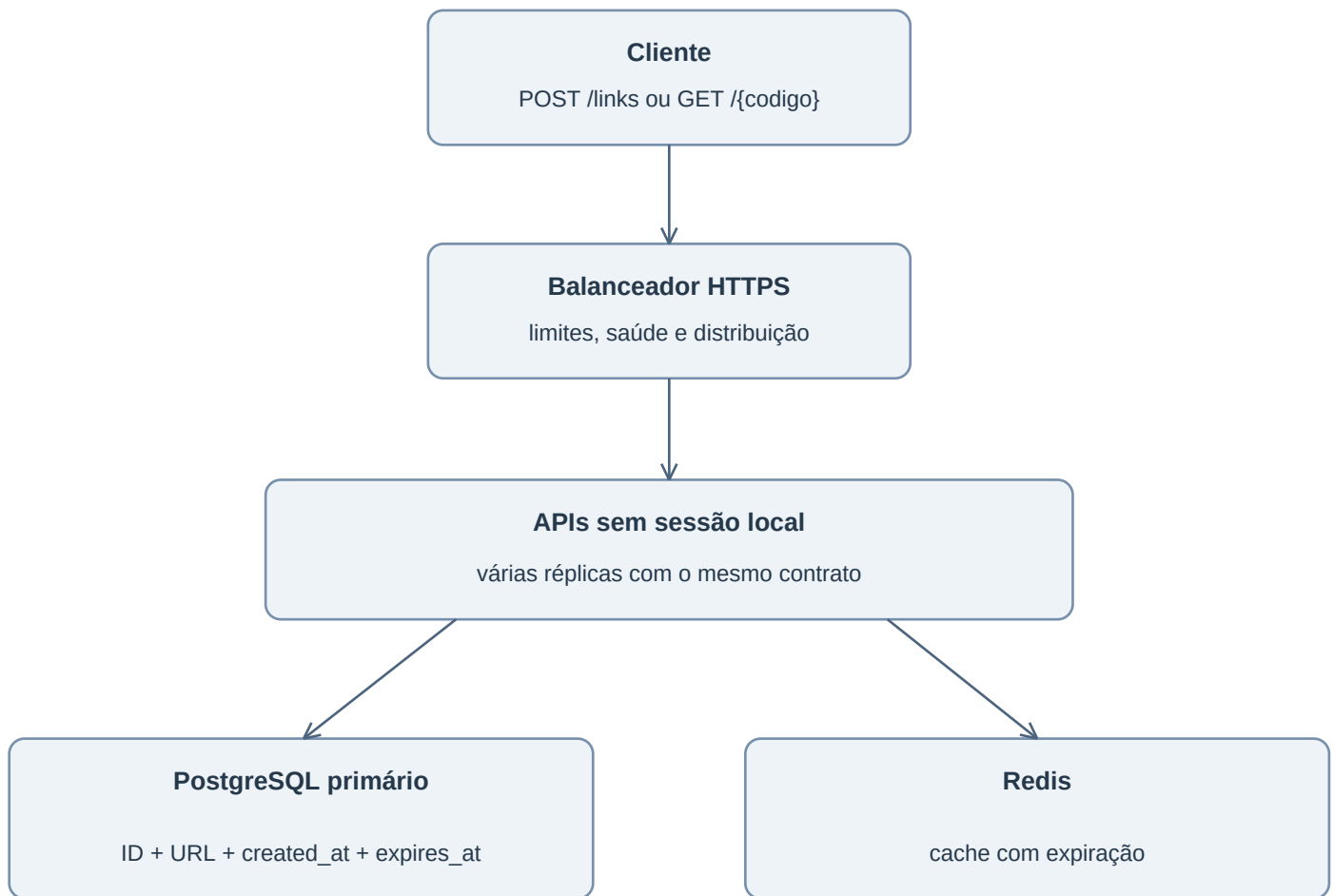


Encurtador de URLs

Proposta de arquitetura para gerar códigos únicos, atender muitos acessos simultâneos e controlar a expiração dos links. O desenho é conceitual; a capacidade deve ser medida em teste de carga.



Criação sem colisão

O `POST` valida uma URL `HTTP/HTTPS` e grava a URL original em uma transação. A tabela usa `BIGINT GENERATED ALWAYS AS IDENTITY` e `PRIMARY KEY`. Depois do commit, a API converte o ID para base 62 e devolve o código. A conversão é bijetiva: IDs diferentes produzem códigos diferentes. A chave primária continua sendo a proteção final contra duplicidade. A sequência não deve ser reiniciada e IDs expirados não são reutilizados.

Redirecionamento

`GET /{codigo}` valida o formato e consulta o cache. Em uma falha, decodifica o ID e consulta o banco primário, evitando atraso de réplica logo após a criação. Se o link estiver ativo, responde 302 com Location. Ausente retorna 404; expirado retorna 410. URLs não são buscadas pelo servidor durante o cadastro.

Escala, expiração e verificação

Cache e validade

O registro é imutável após criado. O cache recebe URL e expires_at somente depois do commit. O TTL não pode ultrapassar o tempo restante de validade; a API ainda compara expires_at antes de redirecionar. O banco é a fonte de verdade. Se Redis falhar, a consulta usa o banco com limites de concorrência para evitar sobrecarga.

Muitos usuários simultâneos

As APIs podem ganhar réplicas porque não guardam sessão local. Redis absorve leituras frequentes; conexões ao banco usam pool limitado. Timeouts, limite por cliente e proteção contra rajadas mantêm filas limitadas. Criações passam pelo primário e pela sequência compartilhada. Isso dá unicidade entre processos; não prova uma capacidade específica de requisições por segundo.

Expiração e retenção

expires_at usa UTC. Uma rotina pode remover a URL de links vencidos e preservar o identificador como registro de expiração. Isso permite manter a resposta 410 sem reutilizar o código. Se a política apagar totalmente o registro, o contrato passa a responder 404. Backups e recuperação precisam preservar o avanço da sequência.

Limites e segurança

Base 62 reduz o tamanho, mas não torna um link secreto: códigos sequenciais podem ser enumerados. Esta proposta é para links públicos. HTTPS protege o transporte; limites de tamanho e validação de esquema rejeitam URLs inválidas. Não executar chamadas ao destino no servidor evita transformar o cadastro em um buscador de endereços internos.

Como eu verificaria

Testes concorrentes devem criar milhares de links por várias réplicas e conferir ausência de códigos repetidos. Também verificaria redirecionamento imediato, expiração durante acesso, falha do cache e recuperação do banco. Um teste de carga deve medir latência p95/p99, erros, conexões e saturação até encontrar a capacidade sustentável. Nenhum desses resultados de carga é presumido nesta proposta.

Referências

PostgreSQL: <https://www.postgresql.org/docs/current/ddl-identity-columns.html>

Redis: <https://redis.io/docs/latest/commands/expire/>